Eseményvezérelt alkalmazások

1. Beadandó feladat dokumentáció 2026/2027 őszi félév

Készítette: Sümegi Gergő

E-mail: sumegigergo100@gmail.com

Feladat

4. Billenő labirintus

Készítsünk programot, amellyel a következő játékot játszhatjuk. Adott egy n × n elemből álló játékpálya, amely falakból és padlóból áll. A padlón piros és kék golyók, valamint piros és kék lyukak helyezkednek el. A játékos a pályát a négy irány valamelyikébe billentheti. Ekkor az összes golyó egyszerre gurul a megadott irányba, egészen addig, amíg falnak vagy egy másik, már megállt golyónak nem ütközik. Ha egy golyó a saját színével megegyező lyuk fölé ér, beleesik, és eltűnik a pályáról. Ha más színű lyuk fölé ér, a golyó elakad, és a játékos veszít. A golyók gurulását a program animálva jelenítse meg.

A játékos célja, hogy minden golyót a megfelelő lyukba juttasson, minél kevesebb billentéssel. A játékos a lépéseit tetszőleges számban visszavonhatja. A visszavonás is lépésnek számít.

A pályák méretét és felépítését (a falak, golyók és lyukak helyzete) fájlban tároljuk. A program legalább 3 különböző méretű pályát tartalmazzon. A program pályánként őrizze meg a legjobb eredményt, azaz a legkevesebb billentés számát, amely a program újraindítása után is legyen elérhető. A program biztosítson lehetőséget új játék kezdésére a pálya kiválasztásával, valamint játék szüneteltetésére (ekkor nem telik az idő, és nem lehet billenteni). Ismerje fel, ha vége a játéknak, és jelenítse meg, győzött vagy veszített-e a játékos. Győzelem esetén jelezze, ha a játékos új rekordot ért el. A program játék közben folyamatosan jelezze ki a játékidőt és a billentések számát.

Elemzés

A feladatot egyablakos asztali alkalmazásként, Windows Forms grafikus felülettel valósítjuk meg. A program indításkor automatikusan az első, fájlból beolvasott pályával indul.

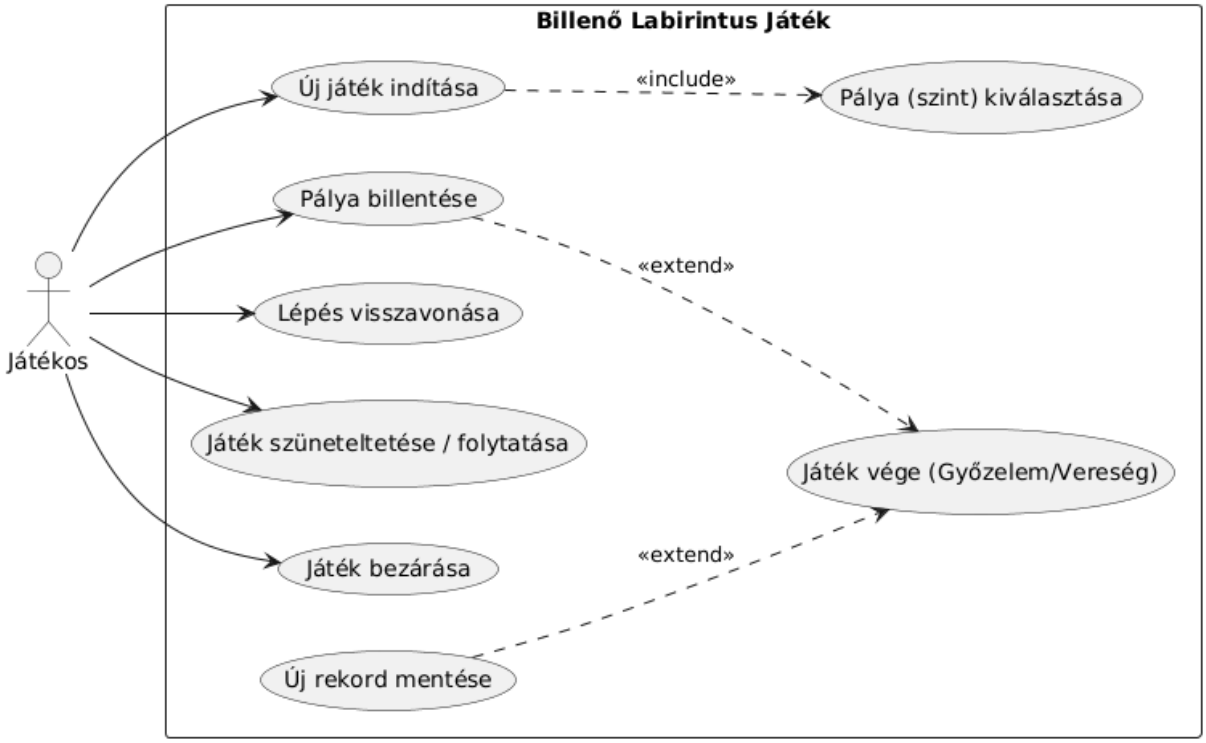
Az ablakban elhelyezünk egy menüt (MenuStrip) a következő menüpontokkal: Game (New Game [Level 1, Level 2, Level 3], Undo, Exit). Az ablak alján egy StatusStrip információs panel folyamatosan jelzi az eltelt játékidőt (timeLabel) másodpercekben, valamint az aktuális lépésszámot (stepLabel).

A játéktáblát egy dinamikusan generált n × n-es rács reprezentálja. Ez egy TableLayoutPanel-ben (gridPanel) elhelyezett PictureBox vezérlőkből (_gridControls) áll, melyek a BuildTable metódusban jönnek létre futásidőben. A játékos a nyílbillentyűk segítségével tudja a pályát a négy irány valamelyikébe billenteni. Ekkor az alkalmazás frissíti a golyók vizuális elhelyezkedését.

Sikeres lépés esetén a program ellenőrzi a golyók ütközését falakkal vagy egymással. Amennyiben egy golyó a színével megegyező lyukba esik, eltűnik a pályáról (inaktívvá válik). Ha rossz színű lyukba esik, a játék vereséggel ér véget. A játékos lépéseit a Memento tervezési minta alapján korlátlanul visszavonhatja (Undo), amely egyúttal növeli a lépésszámot.

A program fájlokban tárolt szöveges állományokból (.txt) olvassa be a különböző szintű pályákat aszinkron módon. A győzelmi rekordokat egy külön állományban (highscores.txt) tartja nyilván. A játék automatikusan véget ér, amikor a modell már nem talál aktív golyót a pályán (győzelem), vagy egy golyó rossz lyukba esett (vereség). A modell ekkor egy GameOver eseményt vált ki, amelyet a Form1 elkap, és egy MessageBox dialógusablakban hirdeti ki az eredményt (jelezve az esetleges új rekordot).

A felhasználói esetek az 1. ábrán láthatóak.



Tervezés

Programszerkezet

A program a követelményeknek megfelelően szigorú háromrétegű architektúrában valósul meg. A megjelenítés, a modell és a perzisztencia logikája külön névterekben helyezkedik el (TiltingMaze, TiltingMaze.Model, TiltingMaze.Persistence). A Nézet (View) réteg (Form1) ismeri a Modell réteget (GameModel), de a Perzisztencia rétegnek csak az interfészét (IMazeDataAccess). A Modell réteg szintén csak az interfészen keresztül kommunikál a perzisztenciával.

Perzisztencia Az adatkezelés feladata a pályák felépítésének betöltése és a rekordok kezelése.

- **IMazeDataAccess (Interfész):** Definiálja a perzisztencia réteg aszinkron "szerződését". Két fő funkciót ír elő: a pálya betöltését (LoadLevelAsync) és a rekordok mentését/olvasását (LoadHighScoreAsync, SaveHighScoreAsync). Stream alapú túlterheléseket is tartalmaz az egységtesztelés megkönnyítésére.

- **MazeFileDataAccess (Osztály):** Az interfész konkrét, aszinkron fájlkezelést használó megvalósítása. A pályákat a levelX.txt fájlokból olvassa ki, és egy int[,] formátumú mátrixba, valamint méretadatokba konvertálja.

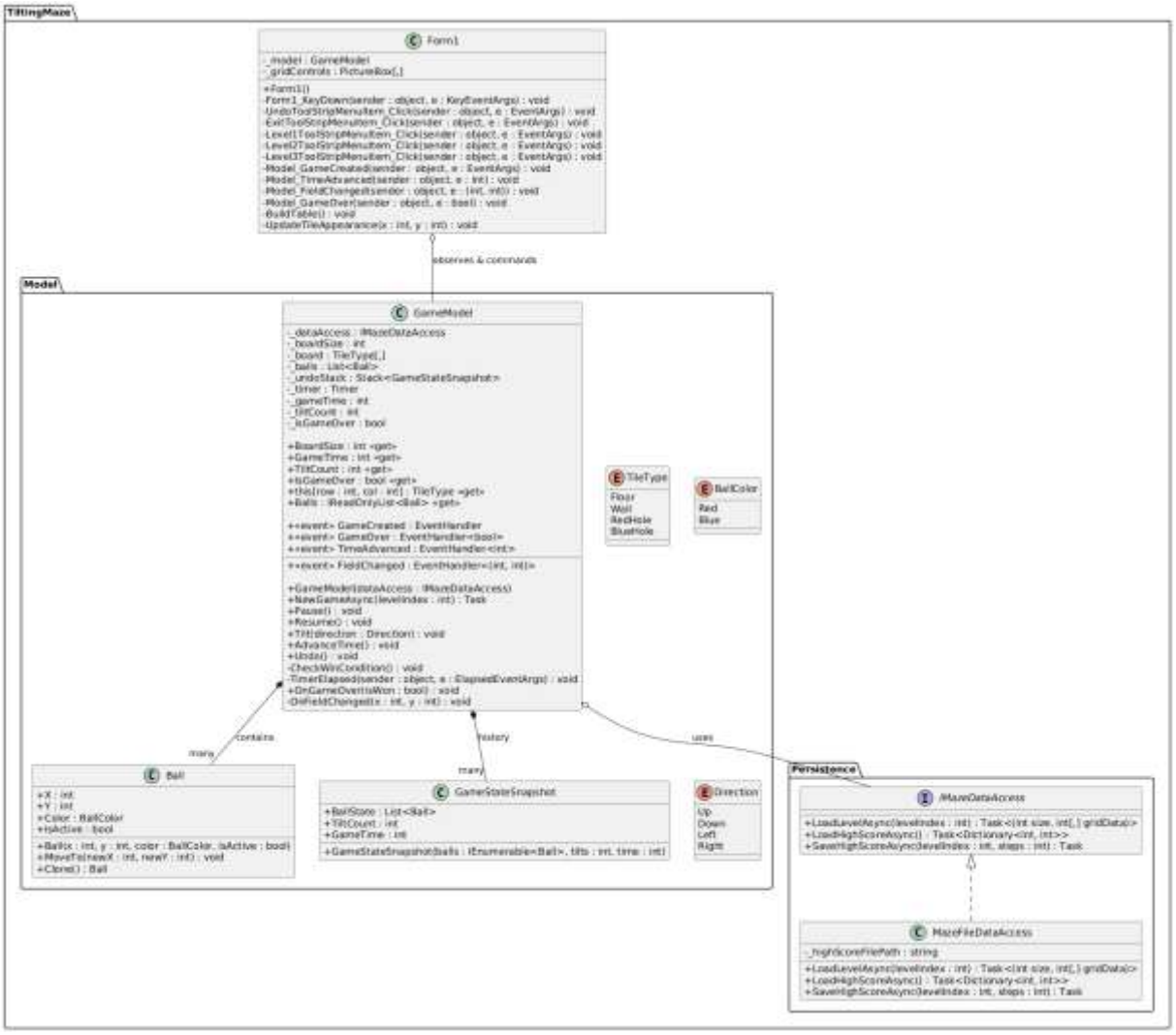
Modell A TiltingMaze.Model névtér tartalmazza a játék üzleti logikáját.

- **GameModel (Osztály):** A modell központi vezérlője. Tárolja a tábla statikus méretét és elrendezését (_board), az aktív golyók listáját (_balls), az eltelt időt és a lépésszámot. Belsőleg egy System.Timers.Timer objektum felel az idő múlásának független, háttérszálon történő számításáért.
- **Ball (Osztály):** Egy mozgó golyót reprezentál, tárolja annak színét, X (sor) és Y (oszlop) koordinátáit, valamint aktivitási állapotát. Rendelkezik egy Clone() metódussal a másoláshoz.
- **GameStateSnapshot (Osztály):** A visszavonás mechanizmusáért felel. A Memento tervezési minta alapján tárolja a golyók deep-copy állapotát (Clone segítségével) és az aktuális lépésszámot/időt a billentés előtti pillanatban. Ezeket a Model egy Stack-ben (_undoStack) gyűjti.
- **Fő metódusai:**
    - NewGameAsync: Adott azonosítójú pálya betöltése, kezdőállapot inicializálása.
    - Tilt: A billentés iránya alapján rendezi a golyókat (a torlódások elkerülése végett), lekezeli a falakkal és egymással történő ütközéseket, valamint ellenőrzi a lyukba esés feltételeit.
    - Undo: Kiveszi a legutolsó mentett állapotot a veremből, és felülírja a golyók listáját, miközben értesíti a nézetet a rajzolási változásokról. Növeli a lépésszámot.
    - CheckWinCondition: Megvizsgálja, van-e még aktív golyó a pályán.
- **Események:** A kommunikáció a Nézet felé a GameCreated, GameOver, TimeAdvanced és a FieldChanged eseményeken keresztül történik.

Nézet A TiltingMaze névtér tartalmazza a felhasználói felületet.

- **Form1 (Osztály):** A nézetet megvalósító fő ablakosztály. Konstruktora példányosítja a fájlalapú adatelérést és a modellt.
- **BuildTable (Metódus):** Futásidőben dinamikusan hozza létre a TableLayoutPanel rácsot és a beleágyazott PictureBox elemeket a pálya mérete alapján.
- **Eseménykezelés:** Feliratkozik a Modell eseményeire. A Model_FieldChanged esemény hatására az UpdateTileAppearance metódus frissíti az adott mező vizuális megjelenését (háttérszín a statikus pálya alapján, golyó színének ráfestése).
- Mivel a Modellben ketyegő időzítő háttérszálon fut, a TimeAdvanced esemény kezelése a Form-ban a BeginInvoke aszinkron hívásba van csomagolva, biztosítva a szálbiztos UI frissítést.

A program teljes statikus szerkezete a 2. ábrán látható.



Tesztelés

A modell funkcionalitása egységtesztek (Unit Tests) segítségével lett ellenőrizve a GameModelTests osztályban, az MSTest keretrendszer használatával. A perzisztencia réteget a tesztkörnyezetben egy MockMazeDataAccess belső osztály helyettesíti. Ez a Mock objektum a valós fájlrendszer helyett előre definiált, a memóriába égetett tesztpályákat ad vissza (pl. azonnali győzelmi, azonnali vereség és akadálypályák), garantálva a tesztek determinisztikus lefutását.

Az alábbi tesztesetek kerültek megvalósításra:

- NewGameAsync_ShouldInitializeCorrectly: Ellenőrzi, hogy egy új játék indításakor a kezdeti állapot (pályaméret, idő, lépésszám, a golyók száma és helyzete) megfelelően áll-e be.
- AdvanceTime_ShouldIncrementTimeAndFireEvent: Az időzítő manuális léptetését szimulálva ellenőrzi az eltelt idő növekedését és a UI-nak szánt esemény kiváltódását.
- Tilt_IntoCorrectHole_ShouldWinGame: Garantált győzelmi szituációt modellez, ahol a golyó belegurul a megfelelő színű lyukba, ellenőrizve az IsGameOver és az eredmény flag-eket.

- Tilt_IntoWrongHole_ShouldLoseGame: Vereségi szituáció szimulációja, amelyben egy golyó szándékosan egy eltérő színű lyuk felé kerül elmozdításra.
- Undo_ShouldRestorePreviousStateAndIncrementSteps: Az állapotmentő (Memento) logika mélyreható tesztelése. Egy mozgás végrehajtása után visszavonja a lépést, és validálja, hogy a golyó valóban visszakerült-e a memóriában az eredeti pozíciójára, valamint az Undo művelet maga növelte-e a lépésszámot a feladatkiírásnak megfelelően.